

Evaluating sparsity on 10000 samples...

Time window: t_min=0.0, t_max=1.0

Model: convnext_tiny, Spiking=True, TTFS_convert=True

Batch [1/67] Accuracy: 97.33% | Avg Layer Sparsity: 33.22%

Batch [10/67] Accuracy: 95.87% | Avg Layer Sparsity: 33.33%

Batch [20/67] Accuracy: 95.87% | Avg Layer Sparsity: 33.33%

Batch [30/67] Accuracy: 95.89% | Avg Layer Sparsity: 33.31%

Batch [40/67] Accuracy: 95.88% | Avg Layer Sparsity: 33.33%

Batch [50/67] Accuracy: 96.03% | Avg Layer Sparsity: 33.33%

Batch [60/67] Accuracy: 95.97% | Avg Layer Sparsity: 33.16%

Overall Accuracy: 95.96%

Overall Average Sparsity: 0.33%

Total samples evaluated: 10000

Papers	Accuracy on Testset (without Fine-tuning) – CIFAR 10 (%)	Sparsity(%)
Nazari et al.	93.6	-
Simonyan (ANN)	93.56	0 (?)
Stanojevic (Nature)	93.59	38
Ours (SNN)- try 1	95.29	33.3
Ours (SNN)- try 2	95.96	33.3

حالت 1 Try :

در این حالت صرفا ConvNext به روش گفته شده در مقاله Nature بصورت اسپایکی پیاده سازی شد به اینصورت که تابع Gelu موجود در ConvNext با حالت اسپایکی خود که بصورت زیر است جایگزین شد.

```

def call_spiking_torch(tj, W, D_i, t_min_prev, t_min, t_max):
    """
    PyTorch version of the TF `call_spiking`.

    Inputs:
        - tj: input spike times tensor (shape [..., C_in]) # last dim is channels/features
        - W: weight matrix (C_in, C_out) or (C_out, C_in) depending on matmul orientation
        - D_i: per-output delay (broadcastable to output shape)
        - t_min_prev: previous layer min spike time (unused for simplified form but kept for
API)
        - t_min, t_max: scalars or tensors

    Returns:
        - ti: output spike times clamped by t_max

    Notes:
        - Ensure matmul dimension ordering matches your usage (here we use `... @ W` with W
shape (C_in, C_out))
    """

    # threshold Eq. 18
    threshold = t_max - t_min - D_i

    # ensure floating dtype
    tj = tj.to(W.dtype)
    threshold = threshold.to(W.dtype)

    # compute ti = (tj - t_min) @ W + threshold + t_min
    # if last dim of tj is C_in and W is (C_in, C_out)
    delta = tj - t_min
    ti = torch.matmul(delta, W) + threshold + t_min

    # clamp to t_max
    ti = torch.where(ti < t_max, ti, t_max)

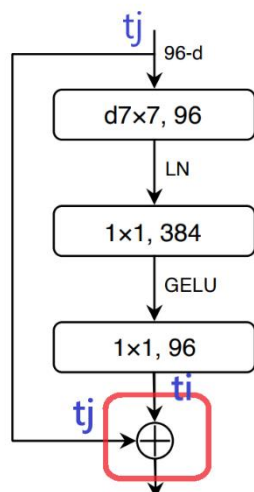
    return ti

```

فرمول شماره 18 جای کار دارد که در ادامه به آن می پردازم. مقدار $D_{\text{ا}}$ در این رابطه نشان دهنده مقدار تغییر است که یک عدد بزرگتر از صفری باید باشد اما فعلاً آن را صفر در نظر گرفته ایم تا اسپایک های ورودی، **بدون تاخیر** در صورت داشتن شرایط اسپایک، اسپایک کنند. وقتی مقدار این پارامتر را به عدد بزرگتر از صفر در نظر میگیریم (آن را Learnable میکنم تا مقدارش در طول فرایند آموزش یاد بگیرد) در اینصورت مقدار Sparsity عوض می شود (البته کمتر می شود که در حال بررسی این موضوع هستیم – مطلوب ما افزایش این مقدار است).

حالت 2 Try :

ConvNeXt Block



در این حالت مقدار $D_{\text{ا}}$ را صفر در نظر گرفتیم تا مقدار sparsity کم نشود که در طول فرایند آموزش روی مقدار 33.3 درصد ماند. تغییرات این مرحله روی بلوک ConvNeXt بود. قسمت residual در مدل اسپایکی نباید جمع بسته بشود. این کار پیام های مهم باید زود تر به لایه بعدی برسد نه دیرتر. با جمع زدن، مقدار t ورودی به بلوک های بعدی بیشتر می شود و این یعنی اسپایک دیرتر.

برای حل این مشکل از کمترین مقدار ممکن t_i و t_j استفاده کردم. بدین صورت که اگر t_j که ورودی به بلوک است، اگر مقدار کمتری نسبت به t_i (مقدار پردازش شده) داشته باشد پس حاوی اطلاعات مهمتری نسبت به t_i می باشد و ما مقدار را انتخاب میکنیم. اما اگر مقدار پردازش شده یعنی t_i کمتر باشد در اینصورت t_i اطلاعات مهمتری دارد. پس خروجی هر بلوک برابر هست با مینیمم مقدار t_i و t_j .

$$\text{Out} = \text{minimum}(t_i, t_j)$$

این کار باعث شد که دقت مدل به مقدار 95.96 درصد برسد